Algorithms for Data Science
Exam 2025
Group **B**

June 2025

Your Name: _____ **Your Student ID:** _____

Question:	1	2	3	4	5	6	7	8	9	10	11	12	13	Total
Points:	5	4	4	6	5	9	5	4	4	5	5	4	0	60
Score:														

The last task gives 4 bonus points. Thus, you can obtain 60+4 points in total (regular+bonus points).

Check if your name and student ID are printed on top of every single page!

Write your name and your student ID on the first page!

Check whether you received all 14 pages (containing 13 tasks in total).

The regular time to solve the exam is 90 minutes.

1. [5 points] Consider the following incomplete Python function. The input is an array $a[0..n-1]$ and a target value x. The array is sorted reversely, that is, $a[0]$ is the largest value in a, and $a[n-1]$ the smallest one.The function outputs the maximum position i of the array such that all values in a from position 0 up to position i have value at least x. In other words, the function returns the maximum i such that, for all $0 \leq j \leq i$, we have $a[j] \geq x$.If $a[0]$ is smaller than x, then the function should return -1.

   Hint: the task is related to the first programming task.

```
1  def compute_max_pos(a, x):
2      n = len(a)
3      left = -1
4      right = n - 1
5      # ??? [Line 5] (Starts with while...)
6          # ??? [Line 6]
7          # ??? [Line 7] (Starts with if...)
8              # ??? [Line 8]
9          else:
10             # ??? [Line 10]
11
12     return left
```

   Complete the code by choosing a correct code snippet for each empty line (that starts with `# ???`). Do it by filling the blanks with the correct labels from the code snippets below:

   - Line 5: **D**
   - Line 6: **G**
   - Line 7: **O**
   - Line 8: **A**
   - Line 10: **I**

   Code snippets to choose from:

   - (A) `right = p - 1`
   - (B) `if x < a[p]:`
   - (C) `while left == right:`
   - (D) `while left < right:`
   - (E) `right = p + 1`
   - (F) `left = p - 1`
   - (G) `p = (left + right + 1) // 2`
   - (H) `p = (left + right + 1) * 2`
   - (I) `left = p`
   - (J) `if x == a[p]:`
   - (K) `right = p`
   - (L) `p = right - left`
   - (M) `while left > right:`
   - (N) `left = p + 1`
   - (O) `if x > a[p]:`

   Note: The indendation of your code is the same as of `# ???`*. In other words, your code for an empty line will start at the position of* `#`*.*

2. [4 points] Write a recurrence for the running time of the following algorithm `AlgoA(`$a[1..n]$`)` in the form of

$$T(n) = aT(n/b) + \Theta(f(n)) \ .$$

You don't need to solve the recurrence! And you don't need to understand what the algorithm computes!

The input of `AlgoA(`$a[1..n]$`)` *is an array* a *of* n *integers.*

The input of the auxiliary function `Copy_Sub_Array(`$a[1..n]$`,`i`,`m`)` *is an array* $a[1..n]$*, a position* i *and a length* m*. The function returns a copy of the subarray* $a[i+1..i+m]$*.*

```
AlgoA(a[1..n])
    if n ≥ 9 then
        ret = 0
        p = n · n
        for j = 1 to (p · p) do
            ret = ret + a[10]
        d = 4 · 2
        m = ⌈n/d⌉
        i = n
        while i > n − 4 do
            c = Copy_Sub_Array(a,i,m)
            ret = ret + AlgoA(c)
            i = i − 1
        for i = 1 to 2 do
            c = Copy_Sub_Array(a,i,m)
            ret = ret + AlgoA(c)
        return ret
    else
        return 10
```

```
Copy_Sub_Array(a[1..n],i,m)
    b[1..m] = new empty array
    for j = 1 to m do
        b[j] = a[i + j]
    return b
```

Your answer: $T(n) =$ $6T(\lceil n/8 \rceil) + \Theta(n^4)$

3. [4 points] Solve the following recurrence using the master theorem. Determine (for yourself) the values for a, b, and $f(n)$ and write down the value for c (as defined in the master theorem). Then, determine which case holds and write the resulting running time. (For case C, you can assume that $f(n)$ satisfies the regularity condition.)

$$T(n) = 10^3 \cdot 10^8 \cdot T(n/10) + \Theta(n^{13})$$

Your answers:

$c =$ **11**

Case: **C**

Resulting running time $T(n) =$ $\Theta(n^{13})$

4. Simplify the following running times as far as possible:

(a) [2 points] $\Theta(\ 3\sqrt{n} + 4 + (n^{5/10+5/10})/2\ ) = \Theta($ n $)$

(b) [2 points] $\Theta(\ 29 \cdot n^{20} + 2 \cdot 11^n + 40 \cdot 10^n\ ) = \Theta($ 11^n $)$

(c) [2 points] $\Theta(\ 3(\log_2 n)^5 + 5 + 1\log_2(n^6)\ ) = \Theta($ $(\log_2 n)^5$ $)$

5. [5 points] Consider the following array $a[1..21]$ containing twenty-one elements:

$$a = \langle 105, 65, 100, 60, 50, 80, 95, 55, 60, 55, 35, 75, 70, 85, 90, 25, 15, 20, 30, 40, 45 \rangle$$

1. Draw the *complete* array as a complete binary tree (as defined for heaps),
2. Mark the edge whose endpoints (parent and child) violate the heap invariant,
3. Choose an appropriate index position j and **decrease** the value of $a[j]$ by the **maximum amount possible** such that a satisfies the heap invariant.

What index j do you choose?
(Write a single number between 1 *and* 21*; there is only one correct answer.)*

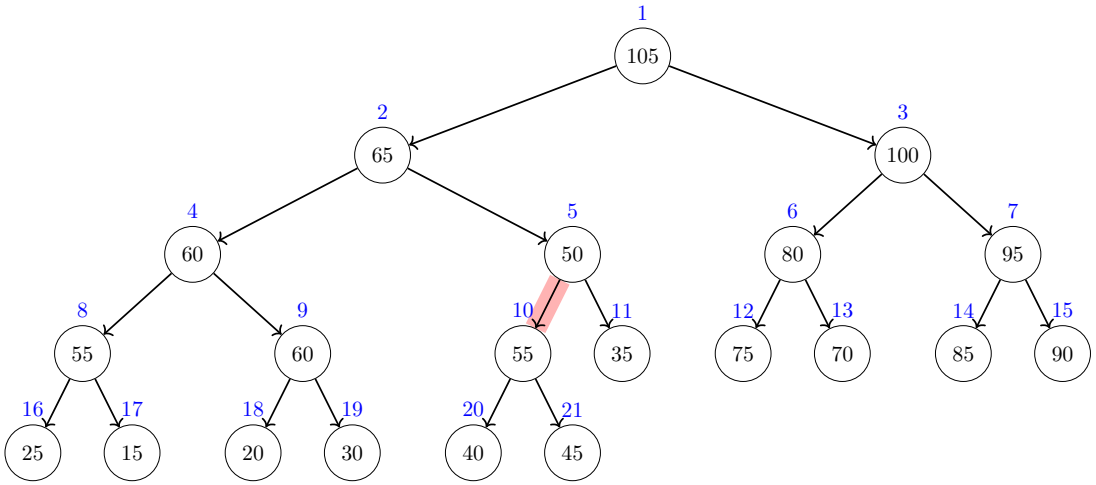
Your answer: $j =$ **10** ***(the violating edge's other endpoint is at* 5*.)***

What is the new value for $a[j]$?
(Write a single number, as small as possible and in particular smaller than the old value of $a[j]$*.)*

Your answer: $a[j] =$ **45** ***(the smallest value from* $[45, 50]$*)***

Your tree hould contain ***all* the elements of the array**, from $a[1] = 105$ up to $a[21] = 45$:

Solution:



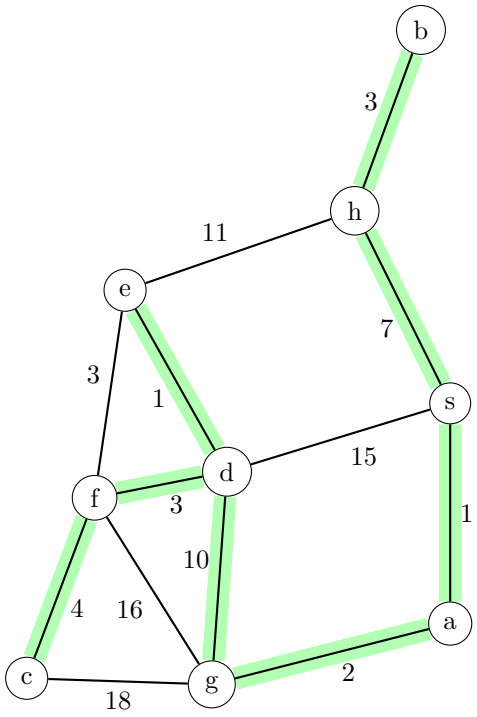
Explanation (not required): The heap invariant is violated at the red edge as the child (index **10***) has a larger value than its parent (index* **5***). To fix the heap invariant, it suffices to decrease* $a[10]$ *to any value between* 45 *and* 50*. The smallest possible value (as asked in the task) is thus* 45*.*

6. [9 points] Dijkstra's algorithm (see below) is run on the following graph where the start node is s.

   What are the values of the *dist* and *parent* tables when Dijkstra's algorithm has finished? Fill in the missing entries in the table below!

   Hint: While solving, write the current distances next to the vertices and mark those vertices that have already been removed from the queue.

```
dist[0..n − 1] filled with INF
parent[0..n − 1] filled with NONE
dist[s] = 0
q=new priority queue
q.add(s,0)
while q not empty do
    v = q.extractMax()
    if v removed for the first time then
        for every edge (v, w) do
            if dist[w] > dist[v] + c_{v,w} then
                dist[w] = dist[v] + c_{v,w}
                parent[w] = v
                q.add(w, -dist[w])
```



Solution:

	a	b	c	d	e	f	g	h	s
distance	1	10	20	13	14	16	3	7	0
parent	s	h	f	g	d	d	a	s	NONE

Note:

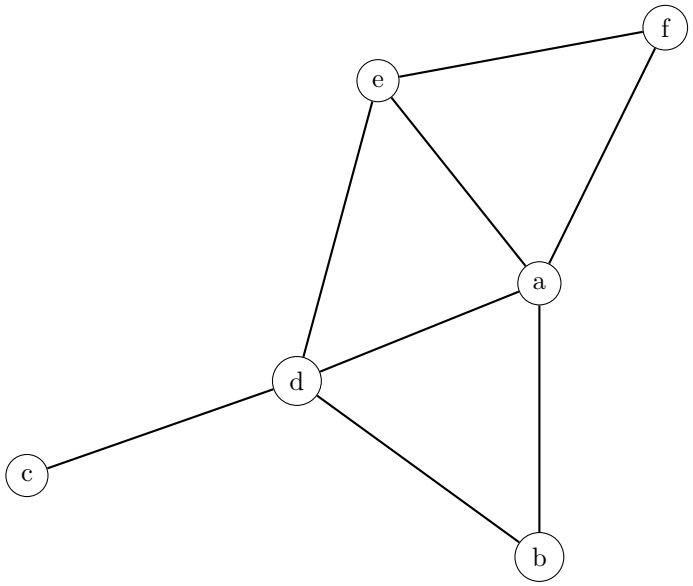
- *The entries of the parent table consist of the names of the nodes or the special value* `NONE`*. For example, the parent of some node could be* s *whereas the parent of* s *is* `NONE`*.*
- *If a vertex has more neighbors, we consider them* ***in lexicographic order*** *(that is, node* a *would come before node* b *and so on)!*

7. [5 points] Draw an (undirected) graph without edge crossings corresponding to the following adjacency matrix. For your drawing, just connect the given vertices below. Your edges don't need to be straight and are allowed to be curves.

   For your score, it's better to draw *all* of the edges even if they cross each other than not to draw one or more edges.

	a	b	c	d	e	f
a	0	1	0	1	1	1
b	1	0	0	1	0	0
c	0	0	0	1	0	0
d	1	1	1	0	1	0
e	1	0	0	1	0	1
f	1	0	0	0	1	0

Solution:



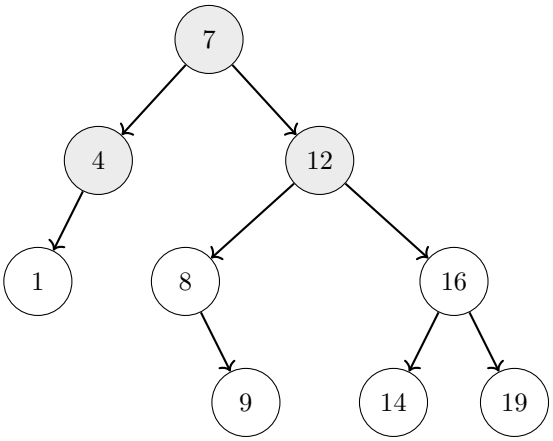
Note that a correct solution has to be crossing free! However, the depicted solution has been automatically drawn by a graph drawing program that does not guarantee that edges do not cross. Thus, if the depicted solution contains edge crossings, keep in mind that such a solution is not fully correct and you would not get all the points by drawing it. However, note that a crossing-free solution can be easily obtained from the depicted solution by redrawing one or more edges as curves around the graph.

8. [4 points] Add the following keys into the following binary search tree **in the order** as they are given below. It suffices that you draw how the binary search looks at the end with each key written in its respective node. (For simplicity, we consider only keys and no values.)

   Note: Do not change connections between previously added nodes! Just add the keys one by one in the given order to their correct positions.

   8, 16, 1, 9, 14, 19

Solution:



9. [4 points] What is the most appropriate data structure for the following scenario, chosen from the list below?

   Unless stated otherwise in the scenario, the goal is to minimize memory usage and the worst-case running time of each operation.

   In your answer, briefly explain how your chosen data structure applies to the scenario. Then give its memory and time complexity, and explain why it performs better than *every* other option, considering the scenario's specific constraints and goals.

   You are implementing the task scheduling subsystem of a web browser. Various subtasks (layout, paint, script execution, etc.) are assigned a dynamic importance score (an arbitrarily large integer), and you must always pick and remove the task with the highest score next. Efficient insertion of new tasks and peeking at the current highest score are critical. Due to the browser architecture, all tasks must be stored exclusively in a single contiguous block of memory.

   1. Array indexed by keys
   2. AVL tree
   3. Find-union
   4. Hash table
   5. Heap/Priority queue
   6. Sorted Linked list
   7. Queue
   8. Stack
   9. Sorted array
   10. Unsorted array

   Most appropriate data structure: **Heap/Priority queue**

   Explanation:

   Solution: We use a binary heap where each element is a task and its importance score serves as the priority; calling `ExtractMax()` returns and removes the task with the highest score.
   The memory usage is best possible as the size of the heap is proportional to the number of elements it contains. Furthermore, a heap stores its elements in an array, that is, in an exclusive contiguous block of memory as required.

   The running time of `FindMax()` is best possible as it takes only constant time. The running times of `Add` and `RemoveMax()` are a little worse with $O(\log n)$ time (where n is the number of elements currently in the data structure), but this is still reasonable fast and the best we can get among all the data structures given in the list.

   All the other data structures are worse under this scenario as they don't provide feasible operations (queue, stack, find-union), take more time in the worst case (AVL tree, hash table, sorted/unsorted array, linked list), are not based on an array (AVL tree, sorted linked list), or take too much memory as the maximum key is very large (array indexed by keys).

   Note that in a standard AVL tree, retrieving the maximum key requires $O(\log n)$ time. Although augmenting each node with its subtree's maximum key would allow $O(1)$ retrieval by inspecting the root, an AVL tree inherently relies on pointer-based nodes scattered arbitrarily in memory. This violates our requirement that all keys be stored exclusively in a single contiguous array.

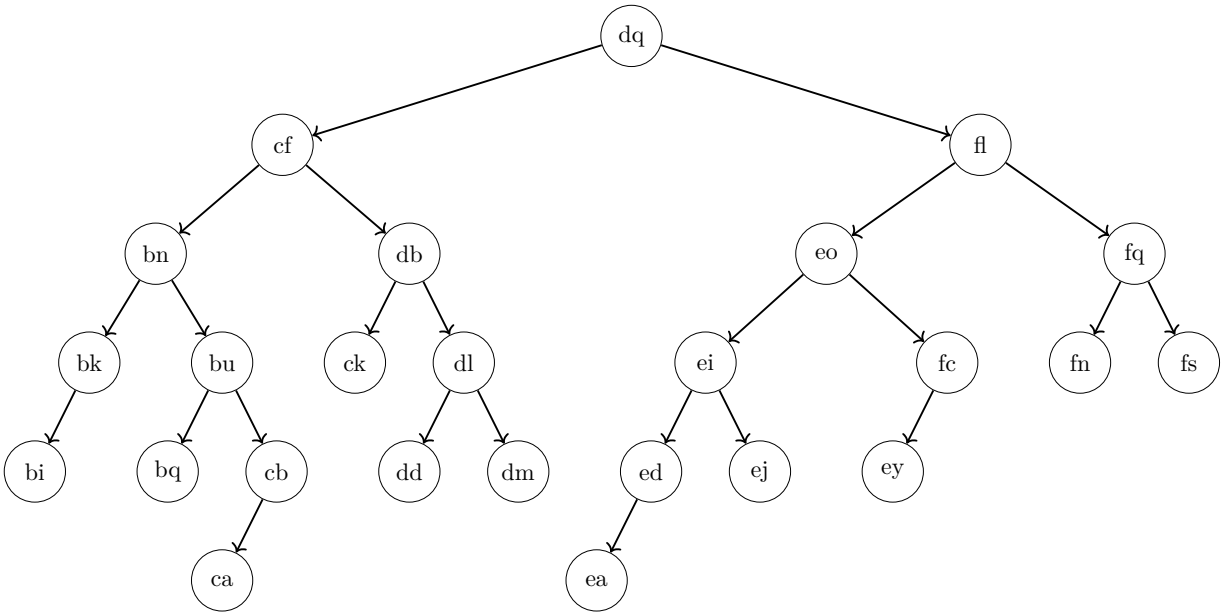
   Note that array indexed by keys is bad in this scenario even if we neglect its memory problem: if M is the maximum key size, it takes $\Theta(M)$ time in the worst-case to find the maximum key.

10. [5 points] What is the height of the node cf in this binary search tree, what is the height of the root (as defined in the lecture and exercises)?

Recall that the height of a node v is the number of nodes *on a longest path starting at v within the subtree of v (not counting the dummy "NONE" nodes that have height* 0*).*

Height of cf: **5** (*Write only a single number.*)

Height of the root: **6** (*Write only a single number.*)



Does the tree satisfy the AVL invariant? Your answer: **no** (*Write "yes" or "no".*)

Write the names of two nodes,

X=**fl** and Y=**fn**, such that the following holds:

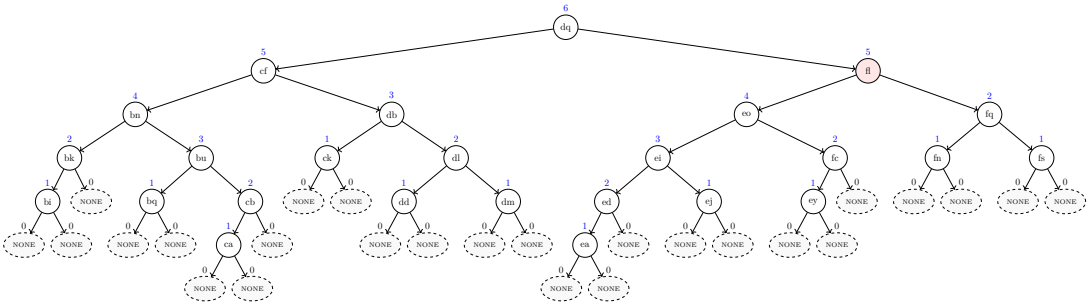
- *If the tree violates the AVL invariant*, then the violation occurs at node X (caused by the heights of X's children), and adding a new child to node Y will restore the AVL invariant.
- *Otherwise, if the tree satisfies the AVL invariant*, then adding a new child to node X will cause a violation of the AVL invariant, and the violation will occur at node Y (caused by the heights of Y's children).
- *If there is more than one valid choice for X*, choose the one with the alphabetically **largest** name. **After selecting** X, if there is more than one valid choice for Y, choose the one with the alphabetically **smallest** name. (*Example: "bi" is alphabetically smaller than "fs".)*

In other words, X is either a node that violates the AVL invariant, or a node where adding a new child would cause such a violation. Similarly, Y is either a node where adding a new child restores the AVL invariant, or it is a node that violates the invariant after a new child is added to X. If there are multiple valid choices for X, choose the one with the alphabetically largest name. If there are multiple valid choices for Y (given your choice of X), choose the one with the alphabetically smallest name.

Note that there is exactly one correct solution that satisfies all the conditions above. It is also possible that $X = Y$.

See next page for an explanation of the solution!

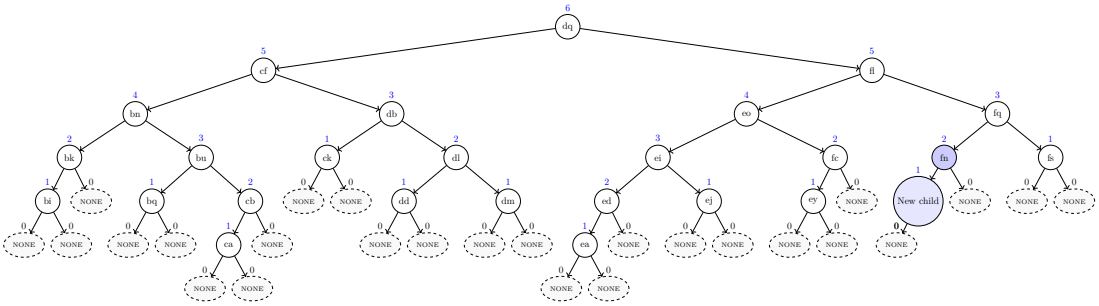
Explanation:
The tree violates the AVL invariant at $\boldsymbol{X}$ = **fl**. The first tree shows the heights and dummy nodes, as well as the violation in red (the parent X whose children differ in height by more than 1).



There are two possibilities to choose a parent where adding a child will fix the AVL invariant:

fn and **fs**.

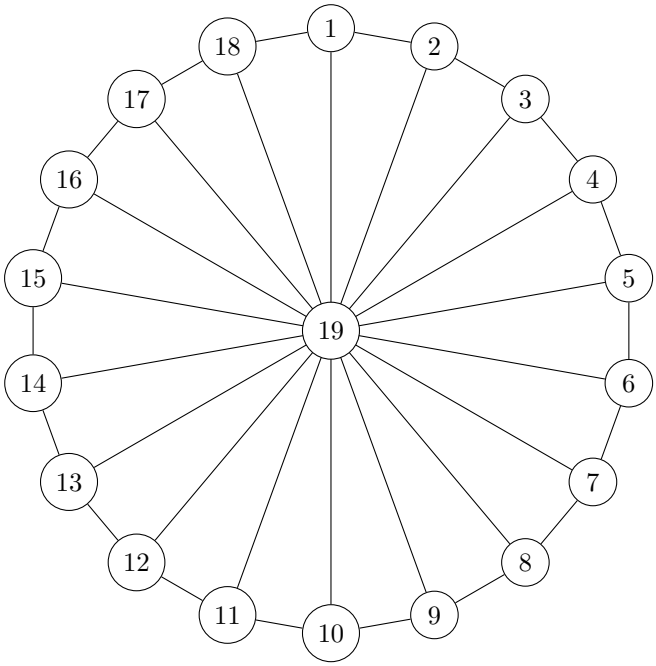
According to the task, you have to choose the parent with the alphabetically smallest label; hence, Y = fn.
The second tree shows the parent $\boldsymbol{Y}$ = **fn** and its new added child (both in blue) where the new child causes the tree to satisfy the AVL invariant.



11. [5 points] Floyd-Warshall's algorithm (see below) is executed on the following huge undirected graph where the vertices are numbered from 1 to n=19 and each edge has the same weight 1. Observe that the vertices 1 to 18 lie on a big cycle and each of them is connected to the center vertex 19 via an edge.

    Unfortunately, the algorithm as written below has an **error** because it stops the main for loop (line 7) already after 12 iterations.

```
1  δ[1..19][1..19] = new 2D array filled with +∞
2  for i = 1 to 19 do
3      δ[i][i] = 0
4  foreach edge (v, w, c) do
       // v and w are the endpoints and c is the weight of the edge
5      δ[v][w] = c
6      δ[w][v] = c
7  for k = 1 to 12 do
8      foreach pair (v, w) do
9          δ[v][w] = min(δ[v][w],   δ[v][k] + δ[k][w])
10 return δ
```



Write the values of the distance table for the following cells as computed by the **erroneous** algorithm above. That is, we still assume that the loop in line 7 iterates **only 12 times**!

Recall that the weight of every edge is equal to 1.

*If the distance is **infinite**, write "INF". Note: There are exactly two fields where the correct answer is "INF". If you write "INF" in more than two fields, you get 0 points for those fields.*

$\delta[7][14] =$ **INF** $\quad\quad \delta[17][12] =$ **INF**

$\delta[11][2] =$ **9** $\quad\quad \delta[14][19] =$ **1**

$\delta[13][18] =$ **13**

12. [4 points] Consider the following algorithm.

```
Input: integer e
u = 4
t = 0
while u < e do
    u = u + 45
    h = −3
    t = t − h
    u = u + h
    t = t − 9
```

Someone wrote the following invariant for the loop:

$$K \cdot u + D \cdot t = F$$

with the three constants

- $K = 2$
- $D = 14$
- $F = 4$

The invariant should hold each time the algorithm evaluates the loop condition. Unfortunately, the invariant has a mistake: one of the three constants, K, D, or F has a wrong value.

Fix the invariant by changing the value of one of the constants!

Which constant do you choose? _____**F**_____ *(Write the name of one of the three constants.)*

The new value for the chosen constant: _____**8**_____

Note: You are allowed to change only one constant.

13. 4 points (bonus)

Consider the following decision problem A:

You are given the floor plan of a museum consisting of rooms connected by corridors. You want to plan a guided tour that starts and ends at the museum entrance, visiting each room exactly once without retracing your steps. Is such a tour possible before closing time?

Consider the following decision problem B:

You manage a server with limited disk space. You have a set of files, each with a given size (in GB) and an importance rating. Can you choose a subset of these files whose total size does not exceed S GB and whose total importance is at least V?

Mark all the correct sentences and give a short explanation.

- √ **A can be reduced to B in polynomial time.**
- √ **B can be reduced to A in polynomial time.**
- ◯ A is in P.
- ◯ B is in P.
- ◯ It is unknown whether A can be reduced to B in polynomial time.
- ◯ It is unknown whether B can be reduced to A in polynomial time.
- √ **It is unknown whether A is in P.**
- √ **It is unknown whether B is in P.**

Solution: Problem A is equivalent to the NP-complete problem Hamiltonian cycle (by interpreting each room as a vertex, each corridor as an edge, and the desired tour as a cycle visiting every vertex exactly once in the resulting graph), and problem B is equivalent to the NP-complete problem Knapsack (by interpreting each file as an item with weight equal to its size, importance as value, storage limit S as capacity, and target importance V as the value threshold).

Both problems can be reduced to each other in polynomial time because

1. both problems are in NP (as they are NP-complete),
2. both problems are NP-hard (as they are NP-complete), and
3. all problems in NP (thus, including A and B) can be reduced in polynomial time to an NP-hard problem (for example, to A and B).

Since it is unknown whether NP-complete problems can be solved in polynomial time, we do not know whether each of the two problems is in P.